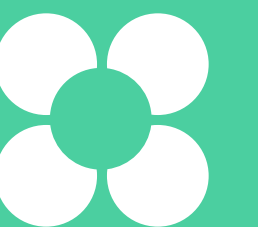


Контекст в Go

Управление временем, отменой и ресурсами
в горутинах

Матвей Ефимов

Разработчик учебных программ и эксперт в Go, Python, SQL



Матвей Ефимов

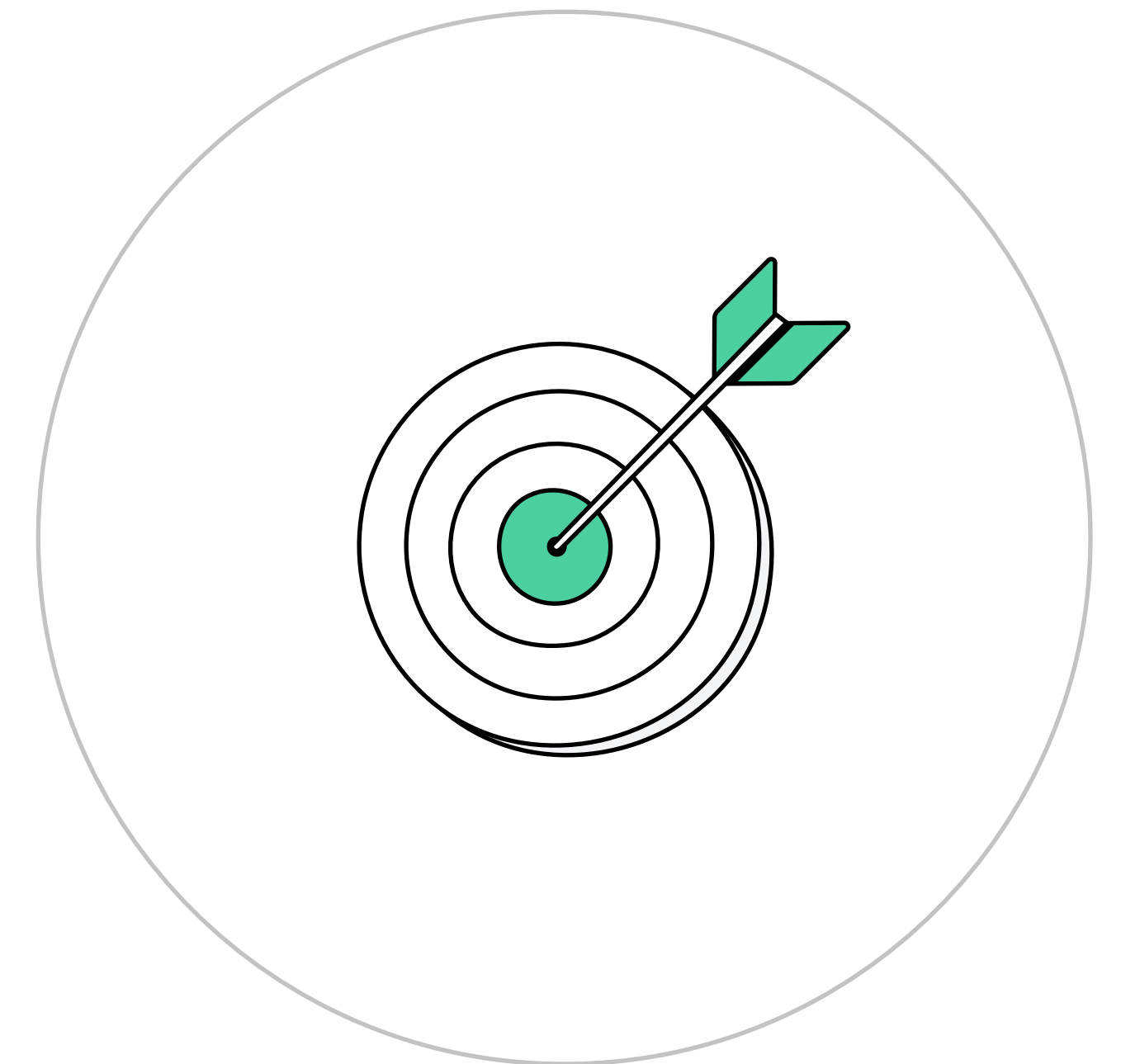
О спикере:

- Преподаватель с более чем 5-летним опытом в IT-образовании
- Разработчик учебных программ и эксперт в Go, Python, SQL
- Фриланс-программист и наставник студентов от школ до вузов



Цели занятия

- Разобрать, что такое контекст в Go и зачем он нужен
- Рассмотреть какие бывают типы контекста
- Как использовать `context.WithCancel`, `context.WithTimeout`, `context.WithDeadline`
- Как контекст помогает избегать утечек горутинов
- Примеры использования с HTTP-запросами и фоновой работой



План занятия

- 1 Зачем нужен контекст
- 2 Базовые типы контекста
- 3 Сигнальный контекст
- 4 Контекст с тайм-аутом
- 5 Контекст с дедлайном
- 6 Практические примеры
- 7 Итоги и чек-лист

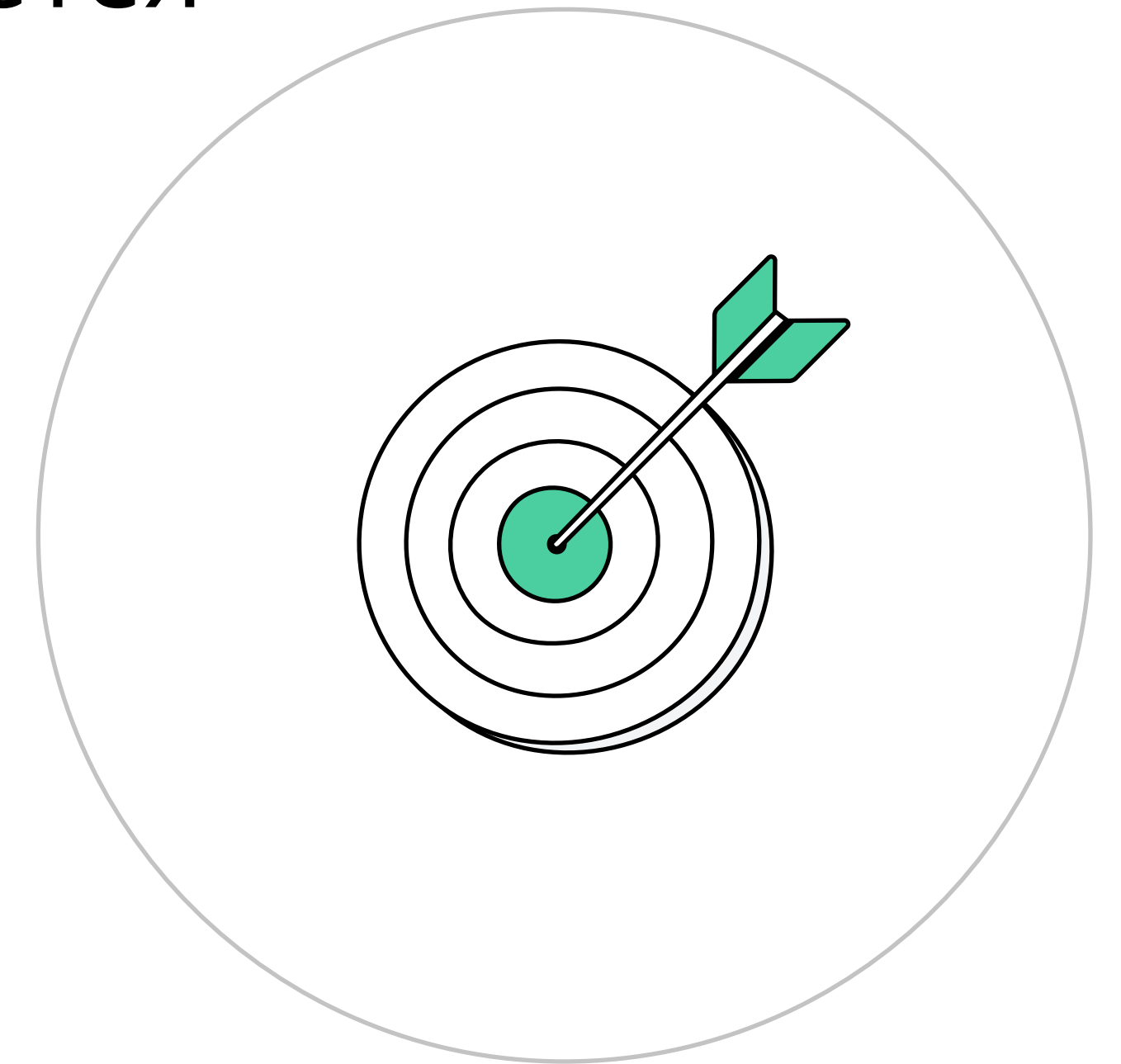


Зачем нужен контекст



Цели видео

- Поймём зачем нужен контекст в Go и какие задачи он решает
- Разберём как контекст помогает управлять временем выполнения и завершением операций
- Рассмотрим типовые ситуации, в которых требуется контролировать жизненный цикл горутин
- Разберём риски, связанные с отсутствием отмены задач и утечками ресурсов
- Узнаем как воспринимать context как стандартный инструмент архитектуры Go-приложений



Что такое контекст

Контекст — это механизм, с помощью которого можно:

- управлять временем выполнения операций
- останавливать ненужные горутины
- отменять операции, если они уже не нужны
- предотвращать утечки ресурсов

Типовая ситуация

Веб-сервер
получил
запрос



Запустил
обработку,
горутины



Клиент
закрыл
браузер



Горутины
продолжают
работать

Почему важно предусматривать отмену задач

Если не реализовать **отмену задач**, фоновые **горутины** будут продолжать работу, даже если они уже не нужны. Это может привести к:

- 1 утечкам памяти
- 2 блокировке соединений
- 3 удержанию файловых дескрипторов

Подобные утечки актуальны не только для HTTP, но и для всех приложений с **параллелизмом**, **сетевыми взаимодействиями** или **долгоживущими задачами**

Зачем нужен контекст в Go

Контекст помогает:

- передать сигнал об отмене
- задать таймаут выполнения
- объединить связанные действия
- завершить фоновые задачи

Когда это нужно:

- при длительных операциях
- в сетевых вызовах
- при работе с горутинами
- для согласованного завершения

ⓘ Горутина узнаёт, что пора остановиться — **через контекст**

Определение контекста в Go

Контекст в Go — это стандартный механизм взаимодействия между компонентами программы. Он используется для:

- передачи **сигнала об отмене**
- **ограничения времени выполнения**
- **распространения данных** между связанными задачами

- Контекст реализован как **канал сигналов** и привычен всем, кто пишет на Go
- Если функция принимает `ctx context.Context`, это означает, что она поддерживает отмену и, возможно, таймаут выполнения

Контекст — инструмент

Контекст — это механизм управления, а не самостоятельная логика.
Он не делает работу за вас, но даёт инструмент, который важно использовать осознанно и корректно

Неправильное использование контекста приводит к ошибкам:

- горутины не завершаются
- `cancel` не вызывается
- течёт память

Если вы не сообщаете горутине, что пора остановиться — вы не контролируете свою программу

Итоги

- Рассмотрели, как работает контекст в Go и какие возможности он даёт
- Разобрали типичные ошибки: работающих «вхолостую» горутин и неосвобождённых ресурсов
- Увидели, почему важно явно передавать и использовать context
- Узнали, что context помогает не только отменять задачи, но и объединять действия, передавать сигналы, работать с тайм-аутами и дедлайнами
- Подчеркнули: контекст — это механизм управления, а не сама по себе логика приложения

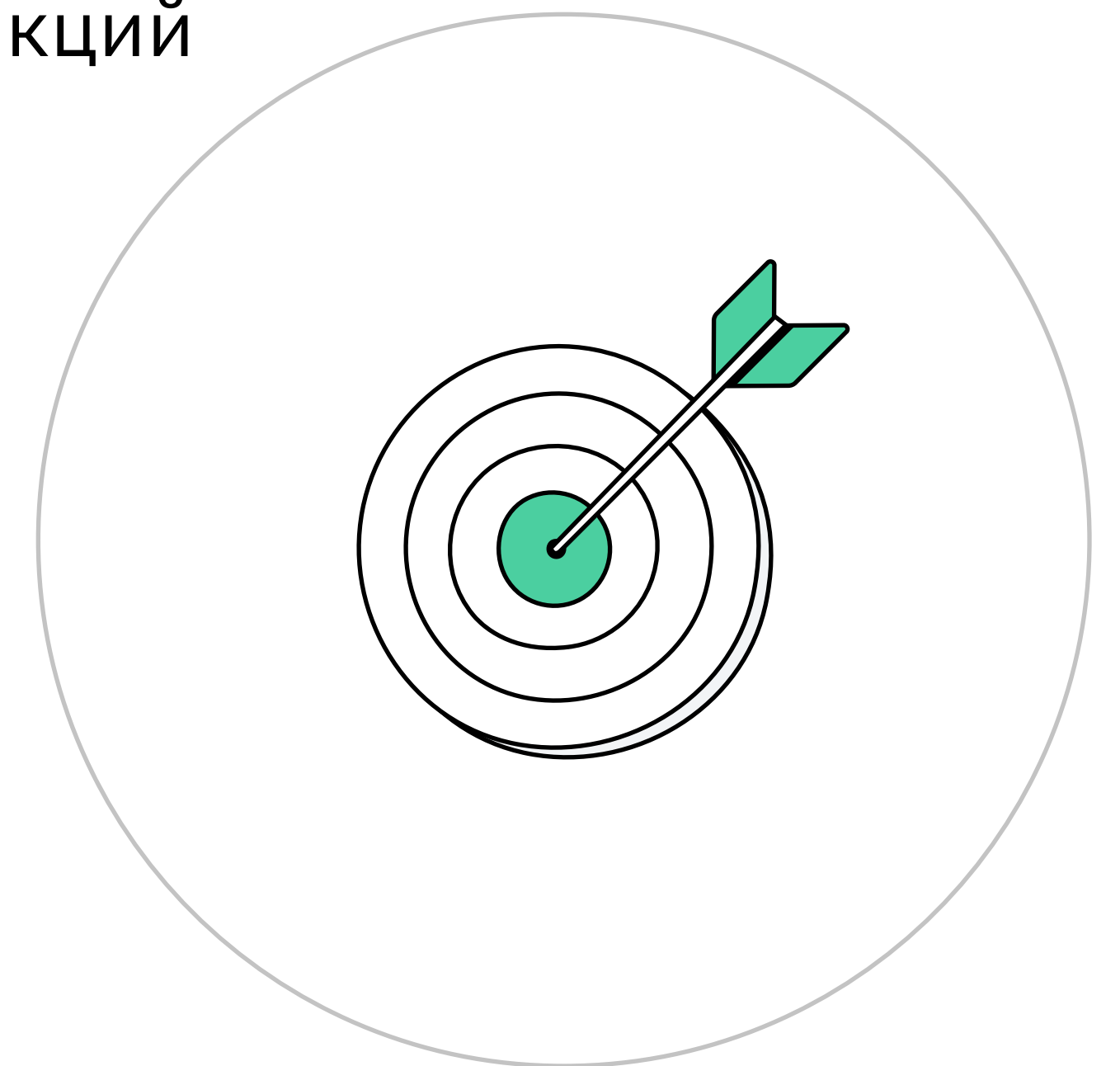


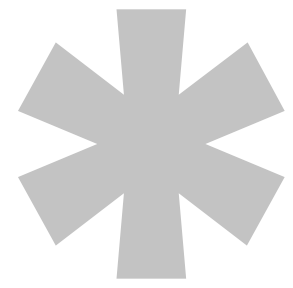
Базовые типы контекста



Цели видео

- Разберём, какие типы контекста используются как основа для отмены и управления выполнением
- Узнаем, что умеет интерфейс Context и какие методы он предоставляет
- Рассмотрим, как использовать `context.Background()` и `context.TODO()` в разных ситуациях
- Разберём, как передавать значения через `context.WithValue` и в чём здесь ограничения
- Узнаем, как правильно передавать контекст по цепочке вызовов функций
- Объясним, почему нельзя использовать строковые литералы в качестве ключей и что делать вместо этого





Контекст необходимо понимать на базовом уровне, чтобы корректно использовать его для отмены операций и ограничения времени выполнения.

Работа начинается с пакета `context` из стандартной библиотеки. Он включает:

- интерфейс `Context`
- базовые реализации

Что такое интерфейс Context

Интерфейс Context определяет базовое поведение, общее для всех реализаций контекста в Go

Он включает четыре метода:

```
type Context interface {  
    Deadline() (deadline time.Time, ok bool)  
    Done() <-chan struct{}  
    Err() error  
    Value(key any) any  
}
```

Эти методы обеспечивают:

- работу с таймаутом (Deadline)
- получение сигнала об отмене (Done)
- проверку ошибки отмены (Err)
- передачу значений по ключу (Value)

Интерфейс и реализация контекста

Context — это интерфейс. Чтобы использовать его в коде, применяются функции из пакета `context`, которые возвращают конкретные реализации.

Одна из базовых функций:

```
ctx := context.Background()
```

Она создаёт пустой **корневой контекст**, с которого обычно начинается работа.

Применение:

- в основной функции приложения
- при инициализации
- в тестах
- в точке входа логики

Важно: `Background()` не передаётся вглубь, а служит для создания других, более «умных» контекстов

Контекст-заглушка

```
ctx:=context.TODO()
```

Возвращает **валидный контекст** для временного использования

Применяется, когда:

- ещё не принято решение, **нужен ли контекст**
- неизвестно, **какой именно контекст** должен использоваться

Это способ явно указать: контекст потребуется, но его тип будет определён позже

Пример

```
func handleRequest() {  
    ctx := context.TODO()  
    doSomething(ctx)  
}
```

Позже `TODO()` можно заменить на `Background()` или на контекст, полученный извне (например, из HTTP-запроса)

Пример использования базового контекста

```
package main

import (
    "context"
    "fmt"
)

func main() {
    ctx := context.Background()
    process(ctx)
}

func process(ctx context.Context) {
    fmt.Println("Обработка началась")
    // эмулируем работу

    select {
    case <-ctx.Done():
        fmt.Println("Контекст отменён:", ctx.Err())
    default:
        fmt.Println("Контекст активен")
    }
}
```

Пример использования базового контекста

Что происходит:

- создаётся базовый контекст `Background()`
- передаётся в функцию `process`
- внутри `process` проверяется отмена контекста через `ctx.Done()`
- так как `Background()` не отменяется, выполняется ветка `default`

Этот пример закладывает фундамент: функция уже поддерживает отмену и сможет реагировать на неё в будущем

Как передавать контекст в функциях

В Go контекст всегда передаётся явно и первым аргументом

Это правило закреплено в стиле языка и поддерживается линтерами

```
func process(ctx context.Context, data string)
```

Неправильно:

- передача через глобальные переменные
- передача не первым аргументом
- хранение в структурах

Передача должна быть прозрачной и последовательной — по всей цепочке: от вызывающей функции к вызываемой, включая горуты, хендлеры, воркеры и т.д.

Контекст как носитель значений

Контекст в Go позволяет передавать не только сигналы об отмене или дедлайне, но и значения, связанные с запросом:

- user ID
- request ID
- **токены авторизации** и др.

Пример записи значения:

```
ctx:=context.WithValue(context.Background(), "userID", 42)
```

Извлечение значения:

```
value:=ctx.Value("userID")  
fmt.Println("User ID:", value)
```

Что нельзя передавать через контекст

Контекст не **предназначен** для хранения произвольных данных.
Его нельзя использовать как общий контейнер.

Допустимо передавать только **технические значения**, нужные на всех уровнях приложения:

- requestID — для логирования
- userID — для авторизации
- traceID — для трассировки

Если бизнес-логика зависит от данных из контекста — это, как правило, **архитектурная ошибка**

Как безопасно задавать ключи в контексте

Не используйте строковые литералы как ключи напрямую — это может привести к **коллизиям**

Правильный способ — создать собственный тип:

```
type ctxKey string
const userIDKey ctxKey = "userID"
ctx := context.WithValue(context.Background(), userIDKey, 42)
value := ctx.Value(userIDKey)
```

Как безопасно задавать ключи в контексте

Почему это важно:

- Метод `ctx.Value()` использует сравнение по `==`
- Если другая библиотека тоже использует строку «`userID`» как ключ, возможен конфликт значений

Итоги видео

Базовые типы контекста, на которых строится работа с context в Go:

- `context.Background()` — нейтральная начальная точка
- `context.TODO()` — временный контекст до уточнения логики
- `context.WithValue()` — способ добавить технические метаданные
- Интерфейс `Context` с методами `Done`, `Err`, `Deadline`, `Value`
- Для реакции на отмену и ограничения



Итоги видео

- Разобрали, что интерфейс Context — это стандартный контракт с методами Done(), Err(), Deadline() и Value()
- Узнали, когда и зачем использовать context.Background() как точку отсчёта
- Рассмотрели context.TODO() как временное решение в незавершённой логике
- Показали, как использовать WithValue для передачи технических параметров (userID, requestId и др.)
- Объяснили правило: контекст должен передаваться явно и первым аргументом
- Разобрали способ безопасной работы с ключами через пользовательские типы



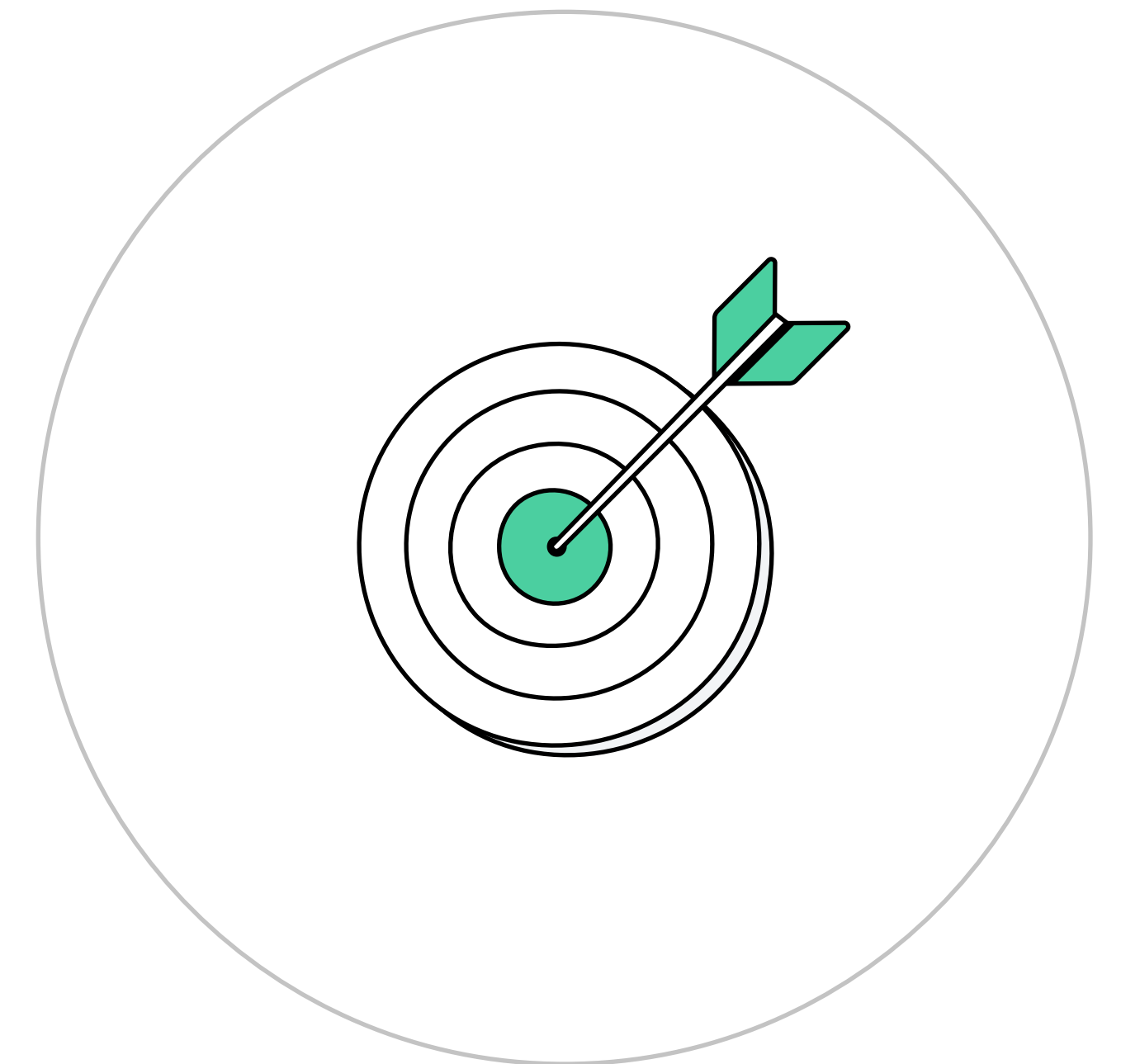
Сигнальный контекст

`context.WithCancel`



Цели видео

- Разберём, как работает `context.WithCancel` и для чего он нужен
- Узнаем, как вручную отменять операции по сигналу из кода
- Посмотрим, как завершать горуты с помощью `ctx.Done()`
- Объясним, почему важно всегда вызывать `cancel()` и к чему приводит его отсутствие
- Разберём, как передавать `cancel` между компонентами
- Поймём, как контекст помогает координировать завершение нескольких горутин



Сигнальный контекст `WithCancel`

`context.WithCancel()` создаёт контекст с возможностью отмены вручную

Применяется, когда:

- 1 отмена должна происходить по сигналу из кода, а не по времени
- 2 завершение зависит от внутреннего события в приложении
- 3 нужно вручную управлять временем жизни операции

Контекст отменяется вызовом возвращённой функции `cancel()`

Сигнальный контекст: общая схема

При вызове `context.WithCancel` создаётся:

- новый контекст, связанный с родительским
- функция `cancel()`, которую можно вызвать вручную

После вызова `cancel()`:

- контекст считается отменённым
- все горутины и функции, использующие этот контекст, могут отреагировать на отмену

Как работает WithCancel

Базовая форма вызова:

```
ctx, cancel := context.WithCancel(parentCtx)
```

После вызова cancel():

- закрывается канал ctx.Done()
- метод ctx.Err() возвращает context.Canceled

ⓘ Это служит сигналом для всех зависимых процессов: **нужно завершить выполнение**

Пример: остановка горютины по сигналу

```
package main

import (
    "context"
    "fmt"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func main() {

    // Создаём контекст, который можно отменить вручную

    ctx, cancel :=
        context.WithCancel(context.Background())

    // Запускаем горутину, которая работает до отмены
```

```
    go func(ctx context.Context) {
        for {
            select {
            case <-ctx.Done():
                fmt.Println("Горютина: получен сигнал
отмены. Завершаюсь.")
                return
            default:
                fmt.Println("Горютина: работаю...")
                time.Sleep(1 * time.Second)
            }
        }
    }(ctx)

    // Ждём сигнала прерывания от пользователя (Ctrl+C)

    sigCh:= make(chan os.Signal, 1)
    signal.Notify(sigCh, os.Interrupt, syscall.SIGTERM)
    <-sigCh
    fmt.Println("main: получен сигнал, вызываем cancel()")
    cancel() // посылаем сигнал отмены

    // Даём немного времени, чтобы горютина завершилась

    time.Sleep(2 * time.Second)
    fmt.Println("main: завершение программы.")
}
```

Разбор кода с WithCancel

1 Создание отменяемого контекста

```
ctx, cancel := context.WithCancel(context.Background())
```

Создаётся контекст ctx и функция cancel() для ручной отмены

2 Запуск горутины с контекстом

```
gofunc(ctx context.Context) { ... }(ctx)
```

Горутинa получает контекст и выполняет работу в цикле

Разбор кода с WithCancel

3 Обработка сигнала отмены

```
cselect {  
  case <-ctx.Done():  
    fmt.Println("Сигнал отмены")  
    return  
  default:  
    // продолжаем работу  
}
```

При закрытии ctx.Done() выполнение прерывается

Разбор кода с WithCancel

4 Ожидание пользовательского сигнала

```
signal.Notify(sigCh, os.Interrupt, syscall.SIGTERM)
```

```
<-sigCh
```

```
cancel()
```

По сигналу Ctrl+C вызывается `cancel()`, и горутина получает уведомление об отмене

Зачем использовать WithCancel

WithCancel позволяет координированно завершать работу нескольких горутин, связанных общей задачей

Пример сценария:

- одна горутина слушает очередь сообщений
- вторая — обрабатывает данные
- третья — записывает в базу

Если в одной из них возникает ошибка или задача становится неактуальной, вызов cancel():

- останавливает всю цепочку
- освобождает ресурсы
- предотвращает утечки и лишнюю нагрузку

Зачем использовать WithCancel

01

Вызов
cancel()



Закрывается
канал ctx.Done()



02

Горутина
читает
ctx.Done()



Получает сигнал
об отмене



03

Внутри select
выполняется ветка
с ctx.Done()



Происходит
завершение работы

Правило при использовании WithCancel

Основное правило: всегда вызывайте `cancel()`, когда контекст больше не нужен

Почему это важно:

- `WithCancel` может выделять ресурсы
- создаёт канал, добавляет слушателей
- без вызова `cancel()` возникает утечка памяти

Итог: если не вызвать `cancel()` — контекст не будет освобождён, а фоновые операции продолжат работать

Вызывайте cancel() с помощью defer

```
func doSomething() {  
    ctx, cancel := context.WithCancel(context.Background())  
    defer cancel() // всегда вызывайте cancel, даже если  
не планируете отмену вручную  
    // используем ctx...  
}
```

Вызывайте `cancel()` с помощью `defer`

Рекомендация:

Сразу после создания контекста вызывайте `defer cancel()`

Зачем это нужно:

- освобождает ресурсы, связанные с контекстом
- защищает от утечек
- работает даже при выходе по ошибке или `return`

Хорошая практика:

`defer cancel()` — обязательная часть шаблона при использовании `WithCancel`

Контекст как сигнал по иерархии

- Контексты образуют иерархию: если родительский контекст отменён, все его дочерние контексты также получают сигнал отмены

Пример:

- `ctx := context.Background()`
- `ctxA, cancelA := context.WithCancel(ctx)`
- `ctxB, cancelB := context.WithCancel(ctx)`
- `ctxC, cancelC := context.WithCancel(ctx)`

Если `ctx` отменяется → `ctxA`, `ctxB`, `ctxC` тоже будут отменены

Зачем это нужно:

- упрощает управление цепочкой операций
- позволяет остановить все дочерние задачи одним вызовом
- устраняет необходимость вручную отменять каждую горутину

Отмену можно вызывать из любой точки

- Контекст можно отменить не только там, где он создан
- Функцию `cancel()` можно передать в другой участок кода — и вызвать позже, при необходимости

Примеры:

- 1 Отмена по таймеру:

```
ctx, cancel :=
context.WithCancel(context.Background())
go worker(ctx)
time.Sleep(5 * time.Second)
cancel() // завершить работу через 5 секунд
```

Отмену можно вызывать из любой точки

Примеры:

② Отмена внутри цепочки вызовов:

```
ctx, cancel :=  
context.WithCancel(context.Background())  
defer cancel()  
doFirstStep(ctx)  
doSecondStep(ctx)
```

Вывод:

- Отмена доступна в любом месте, где есть доступ к cancel()
- Все, кто работает с этим контекстом, получат сигнал и смогут завершиться корректно

Как реализовать функцию с поддержкой отмены

```
func doWork(ctx context.Context) {  
    for {  
        select {  
        case <-ctx.Done():  
            fmt.Println("doWork: получен сигнал отмены")  
            return  
        default:  
            fmt.Println("doWork: работаю...")  
            time.Sleep(1 * time.Second)  
        }  
    }  
}
```

Как реализовать функцию с поддержкой отмены

Что важно:

- Функция принимает `ctx context.Context` как аргумент
- Внутри реализована проверка `ctx.Done()` через конструкцию `select`
- При отмене контекста выполнение корректно завершается

Результат: функция становится управляемой извне и безопасной для многопоточного использования

Итоги видео

- WithCancel позволяет создать вручную отменяемый контекст
- Функция cancel() закрывает ctx.Done() и подаёт сигнал об отмене
- Механизм используется для координации между горутинами
- Все длительные операции должны проверять ctx.Done() и корректно завершаться при отмене
- cancel() нужно вызывать обязательно — это предотвращает утечки ресурсов
- Вывод: WithCancel — основа управляемого завершения фоновых задач и устойчивости систем с параллельной логикой



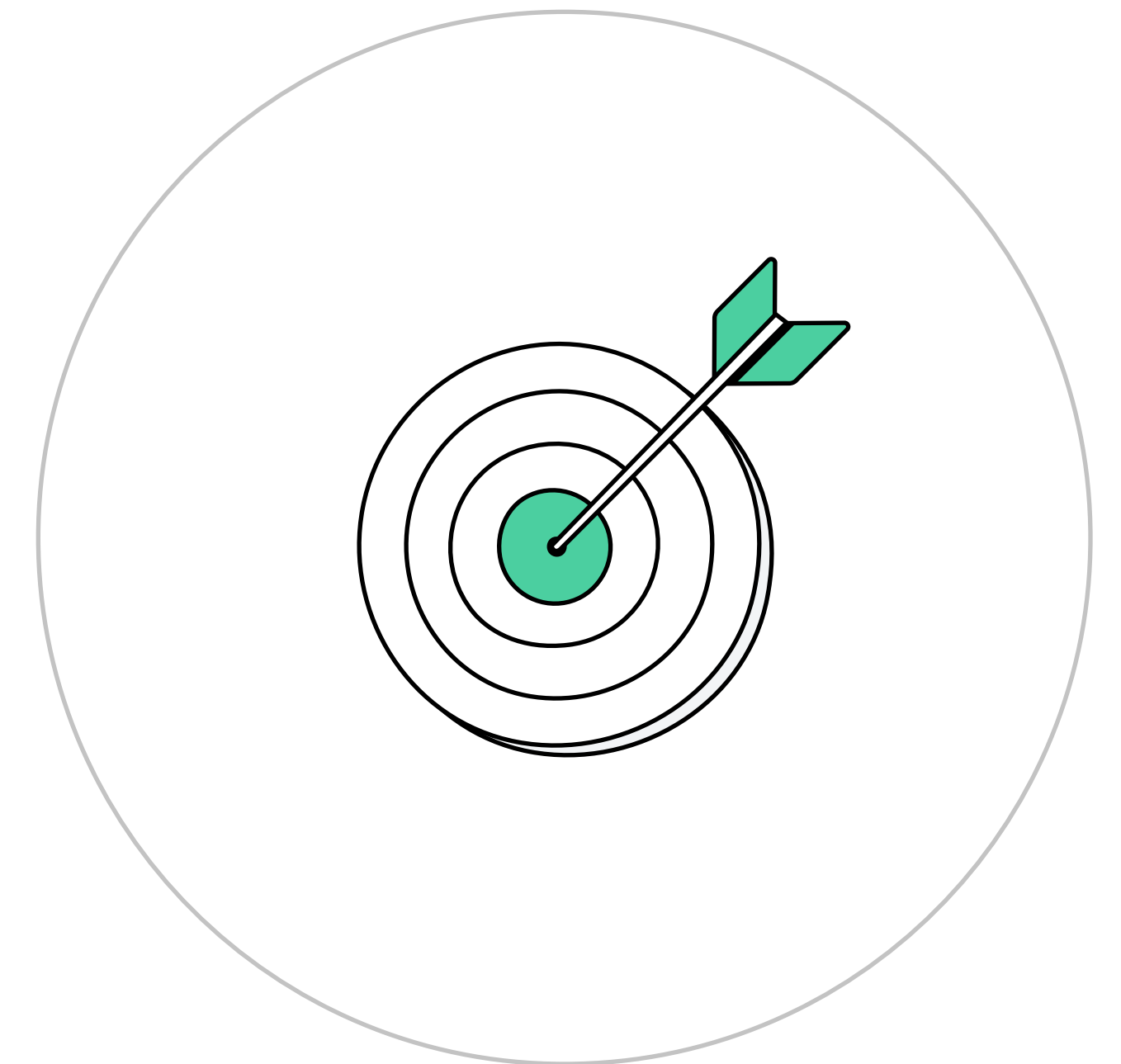
Контекст с тайм-аутом

context.WithTimeout



Цели видео

- Разберём, как работает `context.WithTimeout` и в чём его отличие от `WithCancel`
- Поймём, как задать ограничение по времени на выполнение операции
- Научимся обрабатывать тайм-аут через `ctx.Done()` и `ctx.Err()`
- Поясним, почему важно вызывать `cancel()` даже при автоматическом завершении
- Посмотрим, как использовать тайм-аут в цепочке вызовов и HTTP-обработке



Когда использовать `WithTimeout`

- Иногда отмена может и не понадобиться, но важно ограничить время выполнения операции
- В таких случаях используется контекст с тайм-аутом
`ctx, cancel := context.WithTimeout(parentCtx, timeout)`
- Контекст будет автоматически отменён по истечении указанного времени
- Это позволяет защититься от зависаний и долгих ожиданий без необходимости вручную вызывать `cancel()`

Как работает WithTimeout

- Контекст с тайм-аутом задаёт максимальное время жизни операции
- Если выполнение не завершилось вовремя, контекст автоматически отменяется
- Это особенно важно в ситуациях, где необходим контроль времени:
 - контекст в сетевых вызовах
 - при работе с медленными или нестабильными API
 - в обработке пользовательских запросов
 - в фоновых задачах с ограничением времени

Как работает WithCancel

```
ctx, cancel :=  
context.WithTimeout(parentCtx, duration)
```

- parentCtx — родительский контекст(например, context.Background() или ранее созданный контекст)
- duration — длительность, по истечении которой контекст будет автоматически отменён
- cancel — функция отмены, которую всё равно нужно вызвать вручную, даже если время истекает автоматически

Пример: тайм-аут коротче операции

```
package main
import (
    "context"
    "fmt"
    "time"
)
func main() {
    ctx, cancel :=
context.WithTimeout(context.Background(),
2*time.Second)
    defer cancel() // важно: всегда вызываем
cancel
    result := make(chan string)
```

```
go func() {
    // Эмулируем долгую операцию
    time.Sleep(3 * time.Second)
    result <- "Операция завершена"
}()
select {
case res := <-result:
    fmt.Println("main: получен
результат:", res)
case <-ctx.Done():
    fmt.Println("main: контекст
завершён:", ctx.Err())
}
}
```

Разбор примера с WithTimeout

→ `ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)`
создаётся контекст с тайм-аутом в 2 секунды

→ `defer cancel()`
всегда вызывается для освобождения ресурсов, даже если срабатывает тайм-аут

→ `gofunc() {
 time.Sleep(3 * time.Second)
 result <- "Операция завершена"
}`

горутина «спит» дольше, чем длится контекст — результат не будет получен вовремя

Разбор примера с `WithTimeout`

```
select {  
  caseres := <-result:  
    fmt.Println("main: получен результат:", res)  
  case <-ctx.Done():  
    fmt.Println("main: контекст завершён:", ctx.Err())  
}
```

- если операция не завершится вовремя — выполнится ветка `ctx.Done()`
- Ожидаемый вывод: `main: контекст завершён: context deadline exceeded`

Что происходит при тайм-ауте

Контекст отменяется автоматически, когда истекает указанное время.

Это означает:

- канал `ctx.Done()` закрывается
- метод `ctx.Err()` возвращает `context.DeadlineExceeded`
- все горютины, которые слушают `ctx.Done()`, получают сигнал завершения

Поведение полностью аналогично `WithCancel`, но сигнал подаётся по времени, а не вручную

Пример: функция с ограничением по времени

Код функции:

```
func doWork(ctx context.Context) error {  
    select {  
    case <-time.After(3 * time.Second):  
        fmt.Println("doWork: работа завершена")  
        return nil  
    case <-ctx.Done():  
        fmt.Println("doWork: отмена по контексту:", ctx.Err())  
        return ctx.Err()  
    }  
}
```

Пример: функция с ограничением по времени

Вызов с тайм-аутом:

```
ctx, cancel :=
context.WithTimeout(context.Background(),
2*time.Second)
defer cancel()

if err := doWork(ctx); err != nil {
    fmt.Println("main: ошибка:", err)
}
```

Что произойдёт:

- контекст завершится через 2 секунды
- функция doWork не успеет завершиться самостоятельно
- выполнится ветка ctx.Done() — операция будет прервана

Результат: doWork: отмена по контексту: context deadline exceeded
main: ошибка: context deadline exceeded

Зачем нужен `defer cancel()` при `WithTimeout`

Вопрос:

Если контекст отменяется автоматически по времени, зачем вызывать `cancel()` вручную?

Ответ:

- Чтобы корректно освободить ресурсы, даже при досрочном завершении функции
- `WithTimeout` может выделять внутренние ресурсы
- Без `cancel()` они не освобождаются

Что делает `cancel()`:

- освобождает канал
- останавливает таймер
- снимает слушателей

Вывод: `defer cancel()` — обязательная часть использования `WithTimeout` вне зависимости от того, истекает время или нет

Передача тайм-аутов по цепочке функций

```
func handler(ctx context.Context) {  
    if err := queryDB(ctx); err != nil {  
        fmt.Println("Ошибка при запросе:", err)  
    }  
}
```

```
ctx, cancel := context.WithTimeout(context.Background(), 1*time.Second)  
defer cancel()  
handler(ctx)
```


Передача тайм-аутов по цепочке функций

Что происходит:

- контекст с тайм-аутом передаётся в handler
- все вызовы внутри handler, включая queryDB, ограничены 1 секундой выполнения
- если операция не завершится вовремя — получит ctx.Done() и сможет корректно завершиться

WithTimeout не прерывает выполнение

- Контекст не останавливает функцию принудительно. Он лишь подаёт сигнал через `ctx.Done()`
- Если функция не слушает `ctx.Done()`, она продолжит работать, даже после истечения времени

Что важно:

- контекст — это инструмент координации, а не механизм прерывания
- любая функция, получающая `ctx`, должна реагировать на его завершение

Вывод: без проверки `ctx.Done()` тайм-аут не сработает — всё продолжит выполняться, как будто ничего не произошло

Почему важно слушать `ctx.Done()`

Плохой пример:

```
func badWorker() {  
    time.Sleep(10*time.Second)  
    fmt.Println("badWorker: завершено")  
}
```

- Функция полностью **игнорирует** контекст
- Даже если задать тайм-аут, она всё равно выполнится до конца

Почему важно слушать ctx.Done()

Правильный пример:

```
func goodWorker(ctx context.Context) {  
    select {  
    case <-time.After(10 * time.Second):  
        fmt.Println("goodWorker: завершено")  
    case <-ctx.Done():  
        fmt.Println("goodWorker: отменено:", ctx.Err())  
    }  
}
```

Функция **реагирует на отмену** и может завершиться досрочно

Вывод:

Контекст работает, только если вы проверяете ctx.Done(). Иначе его сигналы не имеют эффекта

WithTimeout в обработчике HTTP-запроса

```
func handler(w http.ResponseWriter, r *http.Request) {  
    ctx, cancel := context.WithTimeout(r.Context(), 5*time.Second)  
    defer cancel()  
    if err := process(ctx); err != nil {  
        http.Error(w, "Обработка прервана", http.StatusRequestTimeout)  
        return  
    }  
    w.Write([]byte("OK"))  
}
```

WithTimeout в обработчике HTTP-запроса

Что происходит:

- тайм-аут ограничивает выполнение process до 5 секунд
- если операция не завершится вовремя — контекст отменяется, и отправляется ошибка 408 Request Timeout

Преимущество: соединение не зависает и не держится дольше допустимого времени

Итоги видео

- `WithTimeout` — способ задать ограничение по времени для выполнения операций
- Контекст создаётся с тайм-аутом и возвращает функцию `cancel`, которую нужно вызывать вручную
- Если операция не завершилась вовремя — контекст отменяется автоматически
- Функции должны слушать `ctx.Done()` и завершаться корректно
- `WithTimeout` применяется при сетевых вызовах, обращениях к внешним API, медленных операциях
- Механизм защищает систему от зависаний, перегрузок и неконтролируемых задач
- Убедитесь, что вы умеете создавать тайм-аут, обрабатывать `ctx.Done()` и не забываете освобождать ресурсы через `cancel()`



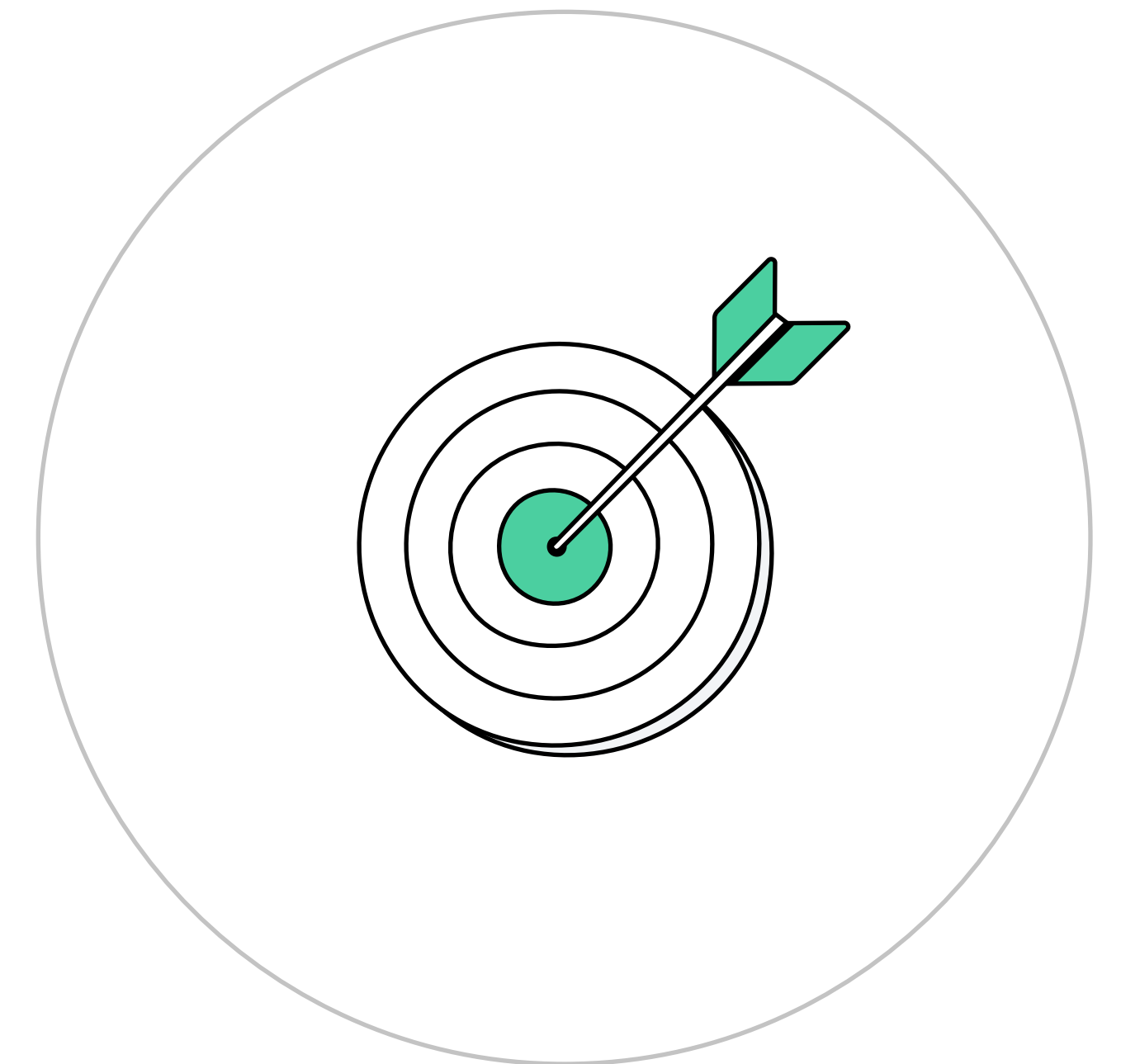
Контекст с дедлайном

context.WithDeadline



Цели видео

- Разберём, как работает `context.WithDeadline` и чем он отличается от `WithTimeout`
- Поймём, в каких случаях удобнее задать абсолютный момент завершения операции
- Научимся создавать контекст с дедлайном и проверять, когда он истечёт
- Посмотрим, как функции реагируют на `ctx.Done()` при достижении дедлайна
- Объясним, как дедлайн влияет на дочерние контексты и как избежать ошибок при переопределении
- Увидим, как использовать дедлайн для управления цепочкой операций с ограниченным временем



Чем отличается `WithDeadline` от `WithTimeout`

- Пример с `WithTimeout`: контекст будет отменён через 3 секунды относительно момента создания

```
ctx1, cancel1 := context.WithTimeout(ctx, 3 * time.Second)
```

- Пример с `WithDeadline`: контекст завершится в конкретный момент времени, независимо от времени создания

```
deadline:=time.Now().Add(3 * time.Second)
```

```
ctx2, cancel2 := context.WithDeadline(ctx, deadline)
```

Что выбрать:

- `WithTimeout` — проще и удобнее в большинстве задач
- `WithDeadline` — полезен, если время окончания уже определено заранее (например, общий тайминг всего запроса)

Когда использовать WithDeadline

Используйте WithDeadline, если:

- контекст должен завершиться в строго заданное время
- например, запрос должен обработаться до 18:00
- время окончания вычислено заранее — по SLA, таймеру или метке времени

Используйте WithTimeout, если:

- нужно ограничить длительность операции
- например, задача должна выполняться не дольше 5 секунд независимо от текущего времени

Вывод:

- WithDeadline — для абсолютного времени
- WithTimeout — для относительной длительности

Пример: контекст с дедлайном

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    // Устанавливаем дедлайн через 2 секунды
    deadline := time.Now().Add(2 *
time.Second)

    ctx, cancel :=
context.WithDeadline(context.Background(),
deadline)

    defer cancel()

    result := make(chan string)
```

```
    go func() {
        // Эмулируем долгую работу –
        4 секунды

        time.Sleep(4 * time.Second)

        result <- "Работа завершена"

    }()

    select {
        case res := <-result:
            fmt.Println("main: получен
результат:", res)

            case <-ctx.Done():
                fmt.Println("main: дедлайн
достигнут", ctx.Err())

    }

}
```

Разбор примера «контекст с дедлайном»

Элемент кода	Комментарий
<code>deadline := time.Now().Add(2 * time.Second)</code>	создаётся момент времени — через 2 секунды от текущего
<code>ctx, cancel := context.WithDeadline(context.Background(), deadline) defer cancel()</code>	создаётся контекст с дедлайном cancel() обязательно вызывается для освобождения ресурсов
<code>go func() { time.Sleep(4 * time.Second) result <- "Работа завершена" }()</code>	горутина «работает» дольше дедлайна — 4 секунды
<code>select { case res := <-result: fmt.Println("main: получен результат:", res) case <-ctx.Done(): fmt.Println("main: дедлайн достигнут:", ctx.Err()) }</code>	выполняется ветка ctx.Done(), так как дедлайн наступает раньше завершения операции

Разбор примера «контекст с дедлайном»

Ожидаемый вывод:

main: дедлайн достигнут: context deadline exceeded

Функция, учитывающая дедлайн

Код функции:

```
func doWork(ctx context.Context) error {  
    select {  
    case <-time.After(3 * time.Second):  
        fmt.Println("doWork: работа  
завершена")  
        return nil  
    case <-ctx.Done():  
        fmt.Println("doWork: отмена  
по дедлайну:", ctx.Err())  
        return ctx.Err()  
    }  
}
```

Вызов с дедлайном:

```
deadline := time.Now().Add(2 *  
time.Second)  
  
ctx, cancel :=  
context.WithDeadline(context.Background(),  
deadline)  
  
defer cancel()  
  
err := doWork(ctx)  
  
if err != nil {  
    fmt.Println("main: ошибка:", err)  
}
```

Функция, учитывающая дедлайн

Что происходит:

- дедлайн установлен через 2 секунды
- doWork выполняется 3 секунды — не укладывается в срок
- срабатывает ctx.Done() — операция отменяется
- функция возвращает ошибку, вызывающая сторона её обрабатывает

Ожидаемый вывод: doWork: отмена по дедлайну: context deadline exceeded main: ошибка: context deadline exceeded

Работа с дедлайнами в коде

Проверка наличия дедлайна:

```
if deadline, ok := ctx.Deadline(); ok {  
    fmt.Println("Дедлайн установлен:", deadline)  
} else {  
    fmt.Println("Дедлайн не задан")  
}
```

Это удобно, если поведение функции зависит от того, задан ли дедлайн в родительском контексте

Работа с дедлайнами в коде

```
if deadline, ok := ctx.Deadline(); ok {  
    fmt.Println("Дедлайн установлен:", deadline)  
} else {  
    fmt.Println("Дедлайн не задан")  
}
```

Переопределение дедлайна: если вы создаёте новый контекст ctx2 на основе ctx1, и задаёте для него дедлайн T2:

- если T2 раньше T1 — будет использоваться T2
- если T2 позже T1 — будет использоваться T1

Контекст не может продлить срок жизни родительского контекста — он может только сократить его

Как избежать ошибок при работе с `WithDeadline`

Рекомендации:

- всегда вызывайте `cancel()`, даже если дедлайн наступит сам — это завершает внутренние таймеры и освобождает ресурсы
- все функции, которые получают контекст, должны слушать `ctx.Done()`, иначе завершение по таймеру не сработает

Отличие в типе ошибки:

- при отмене по дедлайну `ctx.Err()` возвращает `context.DeadlineExceeded`
- при ручной отмене — `context.Canceled`

Как избежать ошибок при работе с `WithDeadline`

Пример диагностики:

```
switch ctx.Err() {  
case context.DeadlineExceeded:  
    fmt.Println("Время вышло")  
case context.Canceled:  
    fmt.Println("Отменено вручную")  
}
```

Это помогает различать тип завершения и адаптировать логику

Общий дедлайн для всей цепочки

Когда важно, чтобы **вся обработка запроса укладывалась в заданный срок**, можно создать один контекст с дедлайном и передавать его во все функции

Пример сценария:

- обработка HTTP-запроса не должна превышать 5 секунд
- в цепочку входят:
 - чтение из базы
 - запрос во внешнее API
 - генерация ответа

Общий дедлайн для всей цепочки

Решение:

- создать ctx с дедлайном на верхнем уровне (например, в `main()` или хендлере)
- передавать этот контекст во все вызовы
- все операции автоматически завершатся, если дедлайн достигнут

Преимущество: согласованное ограничение времени без необходимости задавать таймаут отдельно в каждой функции

Итоги видео

- `context.WithDeadline` — способ задать абсолютный момент времени, до которого операция должна быть завершена
- Поведение аналогично `WithTimeout`: по достижении дедлайна контекст отменяется, канал `ctx.Done()` закрывается
- Функцию `cancel()` нужно вызывать всегда, даже если дедлайн наступает автоматически
- Метод `ctx.Deadline()` позволяет проверять и логировать время завершения
- Ошибка `context.DeadlineExceeded` указывает, что операция не уложилась в отведённое время
- Вывод: `WithDeadline` даёт точный контроль над временем выполнения, особенно в системах с общей временной логикой

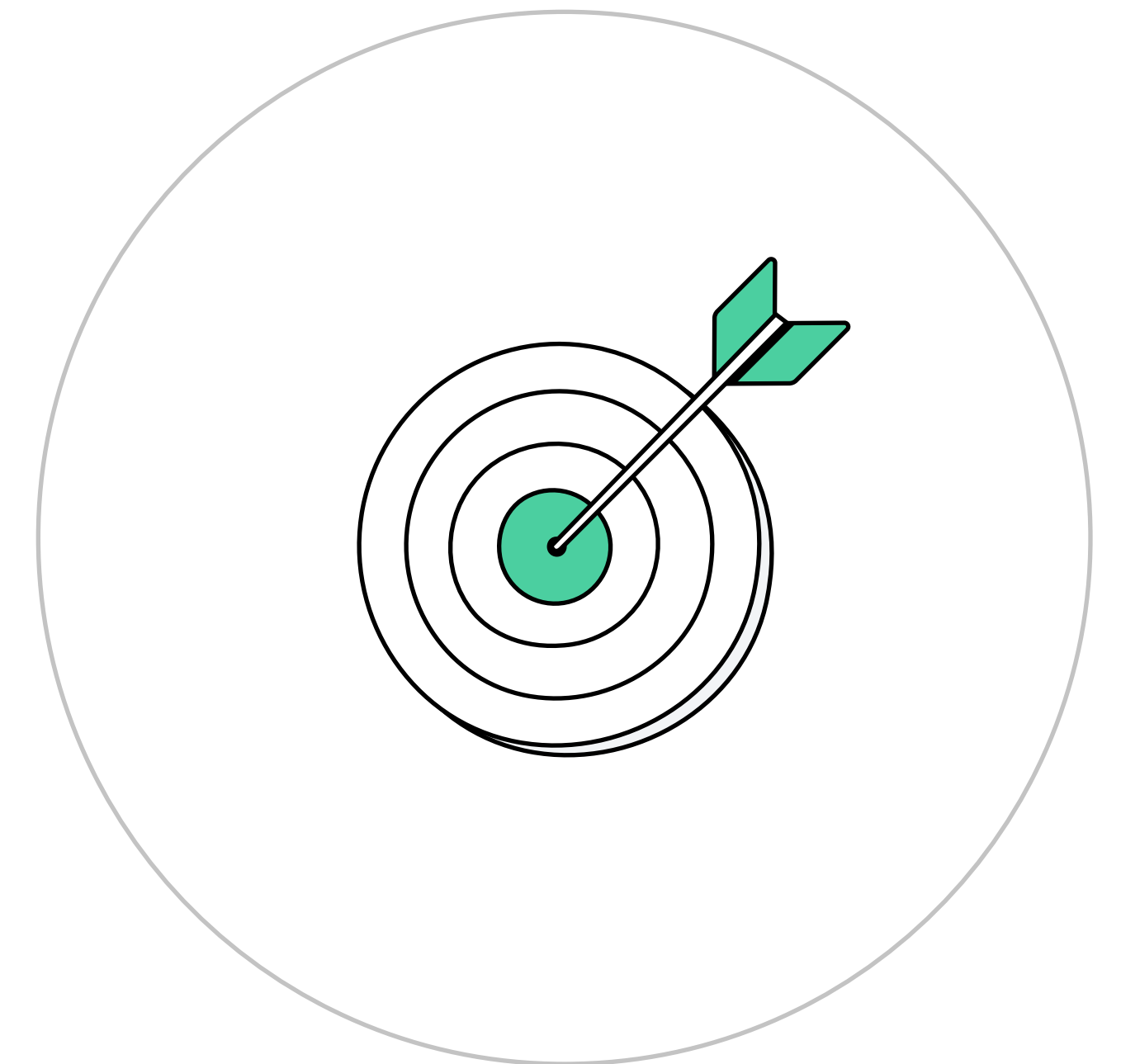


Практические примеры



Цели видео

- Разберём, как использовать контекст в HTTP-сервере
- Посмотрим, как контекст запроса помогает отслеживать обрывы соединения
- Научимся корректно завершать фоновые операции при отмене контекста
- Увидим, как встроить `ctx.Done()` в пользовательские функции
- Поймём, как комбинировать `r.Context()` с `WithTimeout()` или `WithCancel()`
- Закрепим основные приёмы работы с контекстом на реальном кейсе



Кейс: как контекст помогает при обрыве соединения

Ситуация:

Клиент отправляет HTTP-запрос на Go-сервер

Сервер:

- выполняет тяжёлую операцию (например, обращение к базе или внешнему API)
- обрабатывает результат и отправляет его клиенту

Проблема:

Клиент прерывает соединение до завершения операции. Если не учитывать это, сервер продолжает работу впустую: ресурсы не освобождаются база нагружается без необходимости время тратится на ненужную обработку

Кейс: как контекст помогает при обрыве соединения

Решение:

Каждый входящий HTTP-запрос в Go содержит встроенный `context.Context`, который автоматически отменяется, если клиент разрывает соединение.

Достаточно передавать `r.Context()` вглубь всей логики

Результат:

Результат: Фоновые операции прерываются автоматически, если клиент отключился — ресурсы не расходуются зря

Практика: завершение запроса по отмене контекста

Цель кейса: создать HTTP-сервер, в котором:

- в обработчике запускается тяжёлая операция
- операция использует context из запроса
- при обрыве соединения операция автоматически завершится

Практика: завершение запроса по отмене контекста

```
package main

import (
    "fmt"
    "log"
    "net/http"
    "time"
)

func main() {
    http.HandleFunc("/data", handler)
    fmt.Println("Сервер запущен на :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}

func handler(w http.ResponseWriter, r *http.Request)
{
    ctx:=r.Context()
    log.Println("Запрос получен")

```

```
// Создаём канал для результата
    resultChan := make(chan string, 1)
    // Запускаем горутину, эмулируем долгую
операцию
    go func() {
        log.Println("Операция началась")
        // допустим, операция длится 5 секунд
        time.Sleep(5 * time.Second)
        resultChan <- "готовые данные"
    }()

    select {
    case res := <-resultChan:
        log.Println("Операция завершена успешно")
        w.Write([]byte(res))
    case <-ctx.Done():
        // Клиент прервал соединение
        log.Println("Контекст отменён:", ctx.Err())
        http.Error(w, "Запрос отменён клиентом",
http.StatusRequestTimeout)

    }
}
```

Разбор кода: шаг за шагом

Элемент кода	Комментарий
<code>func handler(w http.ResponseWriter, r *http.Request)</code>	Это обработчик HTTP-запроса. Он вызывается при обращении к /data
<code>ctx := r.Context()</code>	Ключевая строка. Мы получаем контекст запроса. Этот контекст автоматически отменяется, если: • клиент закрыл соединение • истёк тайм-аут запроса • сервер закрыл соединение по ошибке
<code>resultChan := make(chan string, 1)</code>	Канал, в который будет помещён результат тяжёлой операции
<code>go func() { time.Sleep(5 * time.Second); resultChan <- "готовые данные" }()</code>	• эмулируем долгую операцию (например, работу с базой) • занимает 5 секунд

Разбор кода: шаг за шагом

Элемент кода	Комментарий
<pre>select { case res := <-resultChan: ... case <-ctx.Done(): ... }</pre>	ждём либо результат, либо отмену контекста если клиент отключился — срабатывает ctx.Done()
<pre>http.Error(w, "Запрос отменён клиентом", http.StatusRequestTimeout)</pre>	возвращаем корректный ответ и не тратим ресурсы зря

Чтобы это проверить на практике

- 1 Запустите сервер: `go run main.go`
- 2 Откройте браузер или терминал: `curl http://localhost:8080/data`
- 3 Сразу же нажмите Ctrl+C, пока не пришёл ответ

В консоли вы увидите:

- Запрос получен Операция началась Контекст отменён: context canceled
- Операция была остановлена ровно в тот момент, когда клиент прервал соединение

Что происходит без контекста

Описание проблемы	Последствия
Если не использовать контекст: Операция продолжает выполняться, даже если клиент отключился	• нагрузка на систему • утечки соединений к базе • замедление отклика для других пользователей • повышенные расходы на обработку неактуальных задач
Контекст — это способ связать жизнь клиента с жизнью горутины. Если клиента больше нет — не должно быть и фоновой обработки	

Если операция сама обрабатывает отмену

Сценарий: функция сама проверяет `ctx.Done()` и корректно завершает работу, если контекст отменён

Пример функции:

```
func longOp(ctx context.Context) (string,
error) {
    select {
    case <-time.After(5 * time.Second):
        return "успех", nil
    case <-ctx.Done():
        return "", ctx.Err()
    }
}
```

Вызов в обработчике

```
result, err := longOp (r.Context())
if err != nil {
    http.Error(w, "Операция прервана",
http.StatusRequestTimeout)
    return
}
w.Write([]byte(result))
```

Если операция сама обрабатывает отмену

Преимущества:

- функция управляет завершением сама
- можно добавлять промежуточные проверки и логирование
- проще контролировать поведение при отмене
- возможность сохранять прогресс или выполнять откат

Как ограничить операцию по времени независимо от клиента

Сценарий:

Иногда важно не только завершить операцию при разрыве соединения, но и задать жёсткий внутренний лимит по времени

Решение:

Создайте обёртку над `r.Context()` с тайм-аутом

```
ctx, cancel :=  
context.WithTimeout(r.Context(), 3 * time.Second)  
defer cancel()
```

Как ограничить операцию по времени независимо от клиента

Что это даёт:

- операция завершится при обрыве соединения
- и автоматически прервётся через 3 секунды, даже если клиент всё ещё на связи

Применение: для защиты от «подвисших» внешних сервисов, долгих вычислений, перегрузок

Как задать внутренний лимит времени в HTTP-обработчике

Вы хотите, чтобы операция завершалась:

- при обрыве соединения (через `r.Context()`) и одновременно
- по жёсткому тайм-ауту, даже если клиент всё ещё на связи

Решение: создайте производный контекст с тайм-аутом

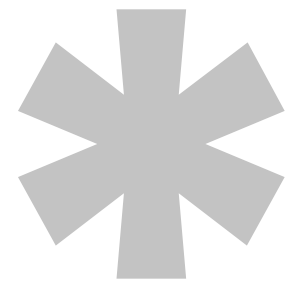
```
ctx, cancel :=  
context.WithTimeout(r.Context(), 3 * time.Second)  
defer cancel()
```

Что это даёт:

- операция прерывается, если соединение разорвано
- и гарантированно завершается через 3 секунды, даже если клиент всё ещё ждёт

Когда применять:

- при обращении к внешним API с нестабильным временем ответа
- при потенциально долгих вычислениях
- для защиты сервера от «зависаний» и утечек времени



**Мы подключили контекст к реальному
HTTP-приложению — и это дало ощутимый результат**

- освобождаете ресурсы, если пользователь отменил запрос
- делаете систему устойчивой под нагрузкой
- снижаете риск утечек и перегрузок
- строите код в духе Go — явно, безопасно, прозрачно

Контекст — не абстракция, а механизм надёжной архитектуры

Итоги



Итоги занятия

- 1 Разобрали назначение контекста
Вы узнали, что `context` в Go используется для:
 - управления временем выполнения операций
 - отмены горутин и цепочек вызовов
 - передачи технических данных между слоями (например, user ID, trace ID)

Контекст помогает завершать работу по сигналу, экономить ресурсы и повышать надёжность приложения



Итоги занятия

2

Рассмотрели базовые типы контекста

- `context.Background()` — стартовая точка для других контекстов
- `context.TODO()` — временная заглушка
- интерфейс `Context` — включает методы `Done()`, `Err()`, `Deadline()`, `Value()`

→

Также разобрали:

- как передавать контекст в функции
- как использовать его в цепочках вызовов
- какие соглашения приняты в Go (контекст — первый аргумент)



Итоги занятия

3 Разобрали отмену операций через WithCancel

- посмотрели, как создавать отменяемый вручную контекст
- увидели, как передавать `cancel()` и вызывать его при необходимости
- рассмотрели механизм передачи сигнала через `ctx.Done()`
- обсудили, почему важно вызывать `cancel()` для освобождения ресурсов

→ Также обсудили:

- как координировать несколько горутин через общий контекст
- как устроена иерархия контекстов и как родитель влияет на дочерние
- где размещать `cancel()` и как писать безопасный код с `defer cancel()`



Итоги занятия

4

Разобрали ограничение по времени через `WithTimeout`

- узнали, как задать время жизни операции через `WithTimeout`
- увидели, что происходит при превышении тайм-аута (`context.DeadlineExceeded`)
- рассмотрели механизм проверки `ctx.Done()` и обработки ошибок

→

Также обсудили:

- зачем вызывать `cancel()`, даже если тайм-аут срабатывает сам
- как использовать тайм-аут в цепочке функций
- чем отличается реакция на отмену от простой задержки в коде



Итоги занятия

5

Рассмотрели контекст с дедлайном (WithDeadline)

- увидели, как задать абсолютный момент завершения действия контекста
- разобрали отличие WithDeadline от WithTimeout: первый – про фиксированное время, второй – про относительное
- узнали, как работает ctx.Err() и какие ошибки возможны при дедлайне
- рассмотрели поведение при вложенных дедлайнах (наследование ограничения от родителя)

→

Также обсудили:

- зачем вызывать cancel() даже при автоматическом срабатывании дедлайна
- как использовать ctx.Deadline() для логирования и отладки
- в каких случаях WithDeadline даёт больше контроля (например, при общей синхронизации всех этапов запроса)



Итоги занятия

6

Разобрали практический кейс отмены HTTP-запроса

- увидели, как `http.Request.Context()` позволяет связать жизненный цикл клиента и сервера
- рассмотрели пример, где серверная операция корректно завершается при обрыве соединения
- узнали, как интегрировать `ctx.Done()` в работу с горутинами и каналами

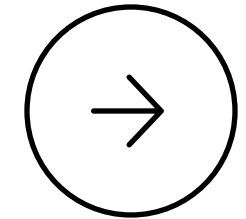
→

Также обсудили:

- как избежать утечек при отмене запроса
- как использовать `WithTimeout` или `WithCancel` поверх `r.Context()`
- как сделать пользовательскую функцию, реагирующую на завершение контекста

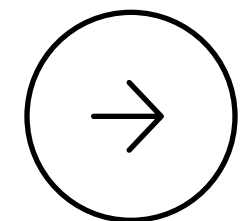


Итоги занятия



Что важно помнить

- контекст не используется для бизнес-логики — только для технического управления
- контекст всегда передаётся первым аргументом, явно
- отмена — это только сигнал: сама функция должна корректно завершиться
- контекст не модифицируется, а порождается заново (через `WithCancel`, `WithTimeout` и т.п.)



Если хотите закрепить

- реализуйте цепочку API-запросов с отменой по тайм-ауту
- добавьте логирование при срабатывании `ctx.Done()`
- используйте `WithValue` для передачи идентификаторов
- изучите использование контекста в популярных библиотеках (`gin`, `grpc`, `gorm`)



Контекст в Go

Управление временем, отменой и ресурсами
в горутинах

Матвей Ефимов

Разработчик учебных программ и эксперт в Go, Python, SQL

